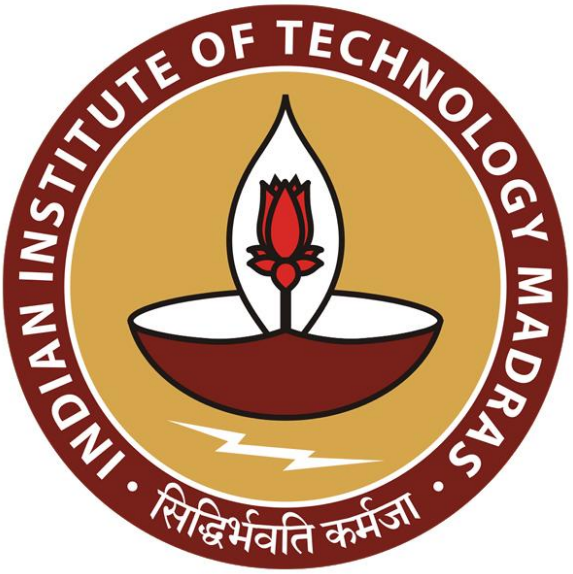
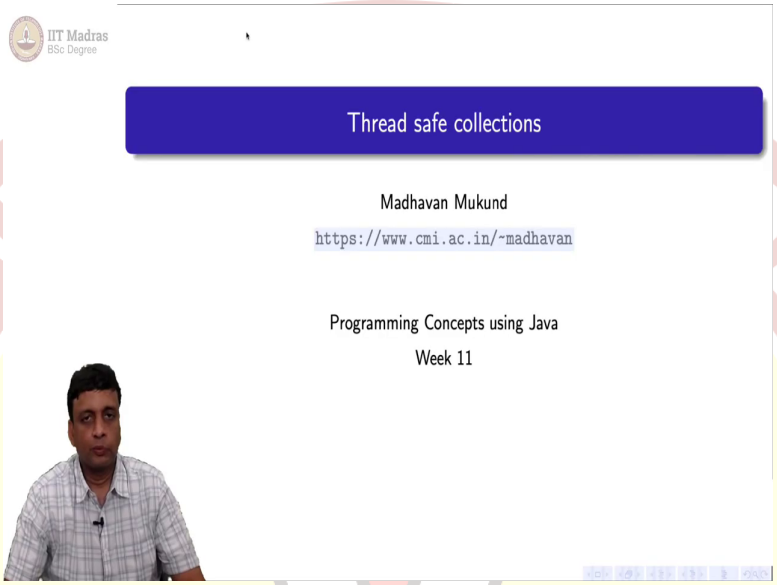


IIT Madras

ONLINE DEGREE

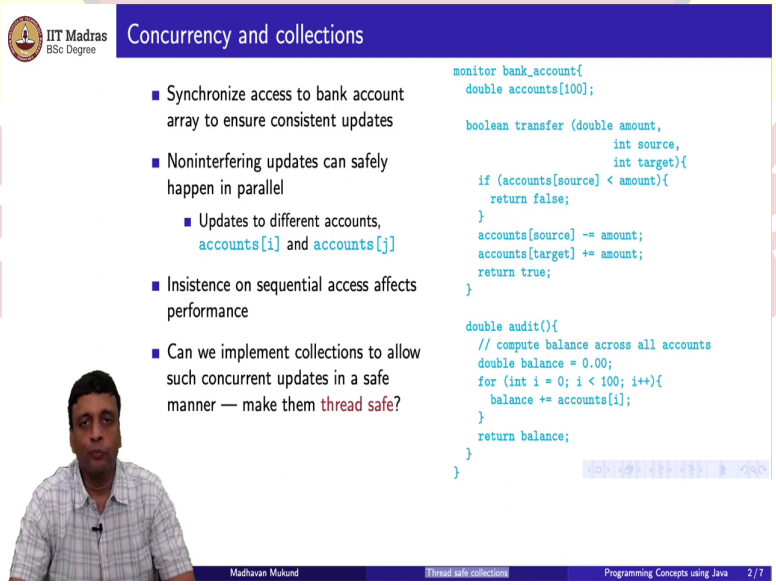
Programming Concepts using Java
Professor. Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology Madras
Thread Safe Collection

(Refer Slide Time: 00:14)



So, we will look at something called thread safe collections.

(Refer Slide Time: 00:17)



So, whenever we were dealing with the example, so far, we often had kind of collective data type. For instance, when we had this bank account example where we wanted to coordinate transferring money between two accounts with an audit thread, which will give us a current

balance across all the accounts, we had stored in an array called accounts, the details of all the bank accounts.

And the whole problem was that we needed to synchronize concurrent access to this collection, in order to make sure that the transfers and the audit thread cooperated and giving us some meaningful output and not some erroneous output where the audit would either interrupt a transfer, or it would report one balance before the transfer and one balance after the transfer.

However, if you just focus on the transfer alone, the transfer alone consists of sending money from one account to another account. So, if I am accessing two separate sets of accounts, those transfers are logically independent of each other. And if I am just focusing on updating one item in this collection, then if that one item that I am updating is different across the thread, so if one thread I am trying to update accounts i, and another thread for a different j, I am trying to update accounts of j, then there is actually no requirement that these two need to be strongly synchronized with respect to each other. Because even if they overlap, the whole problem we had was overlapping.

If I was two of them were trying to access accounts of i, they would read the old value simultaneously. So, they would both get the same old value, and they would write back a new value, but one of the new updates will get lost, because only one of them is updating the original value, the other one should be updating the modified value. But if it is i and j, then the old value of i and the old value of j have nothing to do with each other. So, even if they kind of overlap in time, it does not really matter.

So, if I insist on always locking up in some sense, what we are saying is you take this whole data structure, this accounts array as a whole, and you allow only one thread access to the entire array at a time, regardless of what they are doing, then, of course, you are very safe, because you are making sure that at any given time, only one process or one sequential agent is able to make an update. So, there is never going to be an overlapping inconsistent update.

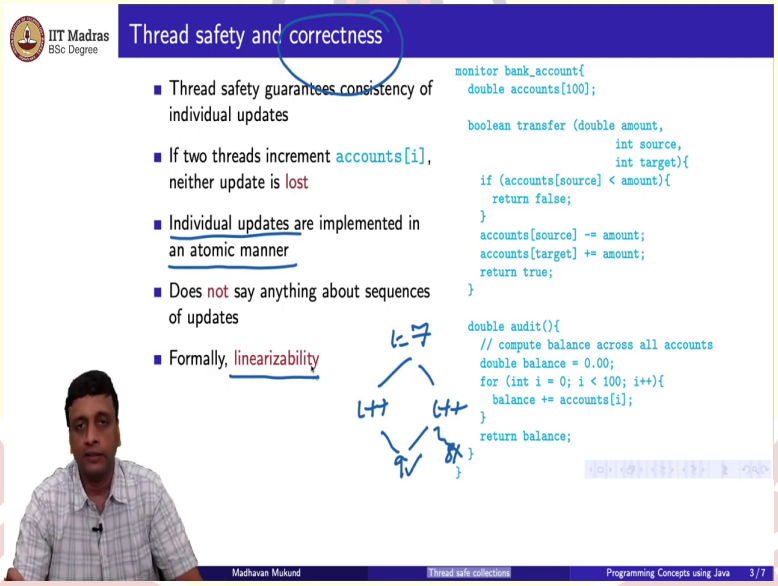
But in a situation like this, where you have non-interfering updates, if you insist that they must happen one after the other, you are potentially reducing the possible performance that you get out of parallel operations. So, you can imagine that if this is a really large bank with

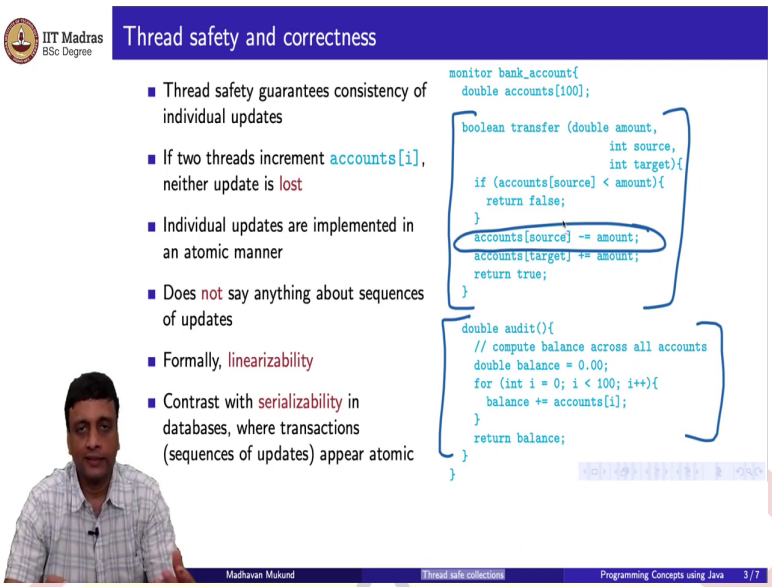
millions of accounts, and there are several people who are trying to access their statements, they are all accessing independently from each other.

Of course, if you try to access your statement from two different computers, I might want to synchronize them. So, you do not see inconsistent values across the two. But if different people are accessing their account from different computers is absolutely no reason why I must force them to wait for the other person to finish.

So, performance-wise, it would be nice if we could actually allow concurrent access to different independent parts of such a data structure. So, how can we implement a collection to allow these threads to access them in parallel in a safe manner whenever safety is something that we decide upon? So, these are what are called thread safe collections. So, how do you make a collection thread safe?

(Refer Slide Time: 03:56)





So, what thread safety will allow us to do is to make sure that independent updates, individual updates of local parts of the data structure are consistent. So, if two threads try to simultaneously update the value accounts, i for a fixed i, then both updates will happen. It will not be that the two updates overlap, and one of them gets lost. Whereas if two things, of course happened in parallel on independent updates like accounts I account see, then there is no question because nothing will affect each other anyway.

So, neither updates should be lost if they access the same data point. So, effectively, what happens is that we want these individual updates. So, we have a collection and the collection consists of these individual points, and we want the individual updates to appear atomic. So, if I am trying to update accounts i and some other thread is trying to update accounts i then our two accounts i update should not interfere with each other.

Now, this does not say anything about sequences of updates. For instance, here I have an accounts source update and an accounts target update where source and target are typically going to be different locations in this array. It is not claiming that these two things will happen together. No, it is only (happy) saying that the update to source, if it is in parallel with some other update to source will not give you an inconsistent result.

And this update to target if it is not in parallel with if it is in parallel with another update to target (())(05:25). But if I have a combination, which affects consistency that I expect some money to flow into the source and then flow out, I could still see various outcomes. So,

formally, this idea about having individual updates, which are implemented atomically. So, this is a notion of correctness.

So, we have to sort of decide what it means for a concurrent application to be, concurrent operation to be correct. So, it means that it must correspond to some real-world thing which we could get without concurrency. That is kind of roughly what it means. So, something can happen in parallel. It is correct. If it looks like it happened one after the other rather than something which happened by criss-crossing.

So, in the sense when I start with a value 7, and then I want to do i plus plus here and i plus plus there. Then I would logically expect after this that the answer is 9. So, if I see an answer 8, it is possible, but it is only possible because of interference and interference is an undesirable thing. So, the correct answer is 9 and the answer 8 is wrong. So, this is formalized in this setting of individual updates made consistent through a definition called linearizability.

So, the important thing is linearizability is referring to individual updates of individual data items inside a collection. Now, in databases, you may have heard the term serializability. So, serializability refers to a transaction. So, if I, for example, I am trying to book a ticket. If I am trying to book a ticket involves several steps. First, of course, I will look up whether the ticket is available, then I will make the booking, then I will make the payment, and this entire process has to complete before the ticket is booked.

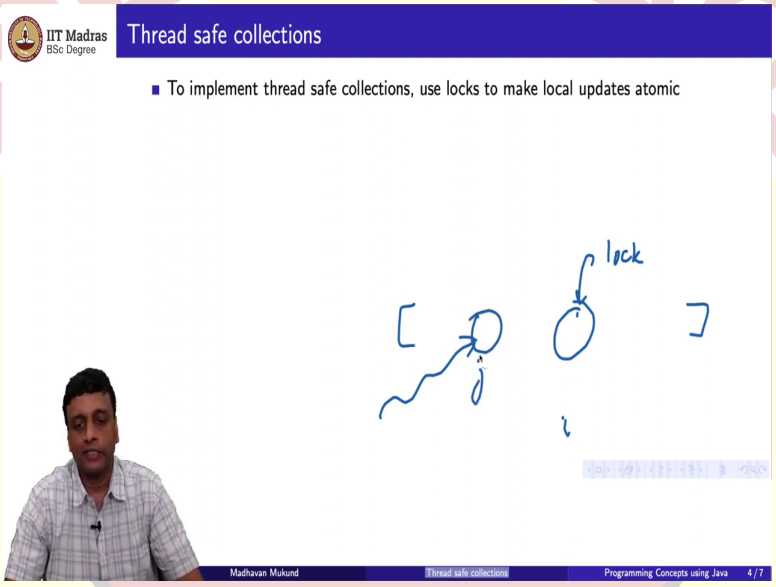
Now, if there is only one available seat on the train, or bus or plane or whatever I am trying to book, then if two people try to book it in parallel, it should not be that both the transaction reached the payment stage and they both pay and then they find it only one of them got it because of course in the final update, only one seat was there. So, they are serializability guarantees that the outcome should look like one person did it before the other person, whichever one we do not care whether the first customer got it or the second customer got it, but they should not both get it.

So, that is the property of the entire sequence, the entire sequence up to the point where the seat is transferred to the name of the customer who got it should appear to be atomic, so that the other thing does not interfere with it. So, serializability is a property of these transactions

as a whole. So, serializability is really what we are saying when we say this whole process of transfer must happen as a unit.

And this whole process of audit must happen as a unit, we are really saying that this entire function must start and finish before it appears that anything interferes with it. Whereas linearizability is just saying that this update to an individual part of the array is guaranteed to happen if I do it in this thread safe way, even if there are other things happening in parallel. So, that is the important distinction.

(Refer Slide Time: 08:23)

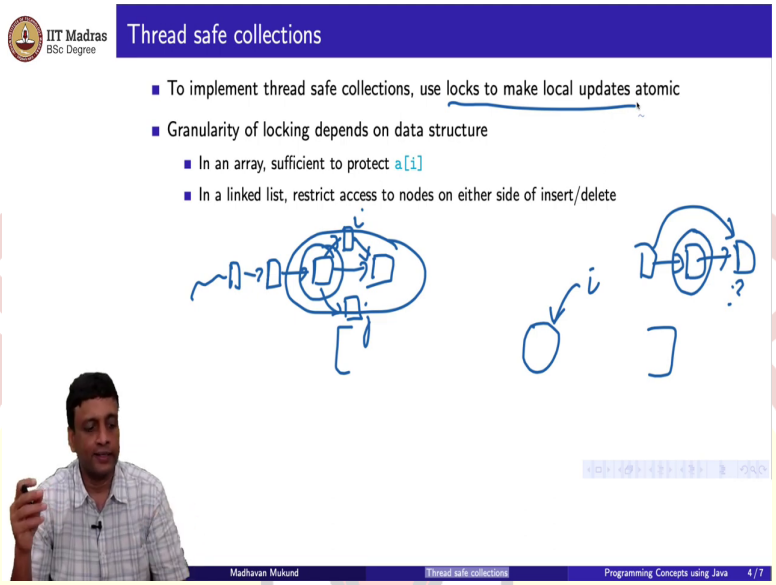


So, the way we implement this is quite straightforward, we will have an array and then if we want to update say the ith position of this array, then we will lock it. So, we saw various ways of locking it, it could be put into semaphore, it could be put into a reentrant lock, we could be having a monitor for this, it does not really matter, we could have synchronized objects. So, in some way we block access to this location, and then we update it.

So, we use a lock to make this local update appear atomic by just simply disallowing other threads access to this till we finish our update. So, the update is not itself atomic, it is the usual thing, I have to look up the value and then change it and put a new value back. But that entire process is protected by some kind of synchronization mechanism.

But that mechanism is restricted to that position, the entire array is not locked up. So, somebody else can come and update another location j without worrying about the fact that i is locked.

(Refer Slide Time: 09:25)



Now, how much to lock depends on the data structure. As we said in an array, if I want to just update ith position, it is enough to lock ith position. But supposing I have a list, a linked list like this. And then I want to insert something at a given position. So, supposing I want to insert something in between these two things. Then what happens is that I need to update the next of the previous and make this point here.

So, I need to also make sure that nobody else is fiddling with this because if they come in insert something else in between, then there will be a contention as to whether the next of the old node is the node that we are trying to insert, or the node that somebody else is trying to insert. So, in this case, we might want to include kind of a neighborhood, at least the previous node.

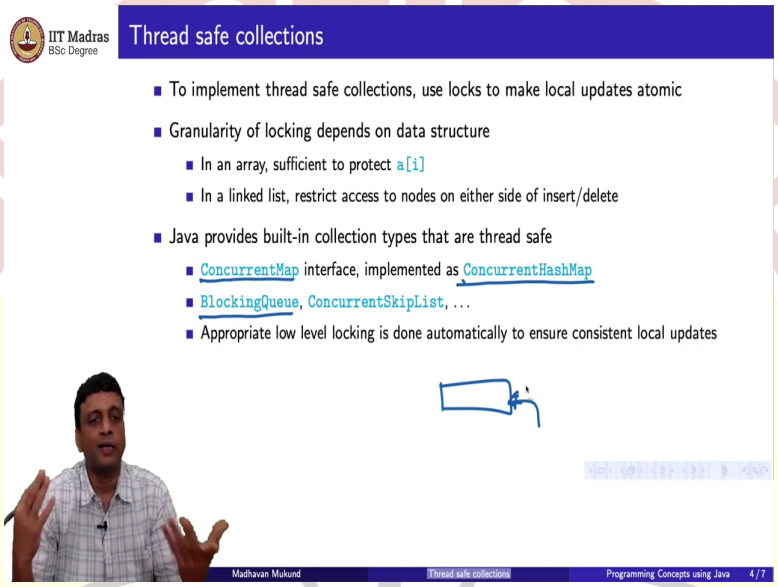
And if I am deleting maybe the next node as well, because if I am deleting a node, for instance, if I have a linked list, and I delete this node, then I want to make this point to that. So, then in this process, I do not want to make things there, suppose this node gets deleted in parallel, then I am in trouble, then this node must point to the node after that, and so on.

So, if it is a depending on the collection, what it means to lock to make the local update atomic may affect just one location in that collection, or it may affect a region, like a

collection might be if it is a tree, it might include the parent and the children, and so on. So, all these things depend on the data structure.

So, it is a real complicated issue to ensure that you have an implementation of a data structure a collection, which supports concurrent access in a safe way, but which maximizes somehow the independent access to different parts of the data structure by concurrent threads.

(Refer Slide Time: 11:09)



So, fortunately, we do not have to do that. So, we can exploit the fact that there are thread safe collections directly because Java has thread safe collections built into the Java dot util dot concurrent library. So, we can directly make use of these in Java, it is implemented under the hood in Java using these local locks in an appropriate way for the different data structures. So, for instance, Java provides an interface.

So, remember, we had a map interface, which gave us the properties of all key value stores. So, now a concurrent map interface, he actually extends the map interface, but it guarantees that two updates that are in parallel to a key will not interfere with each other. So, one, it will appear as though one happened before the other.

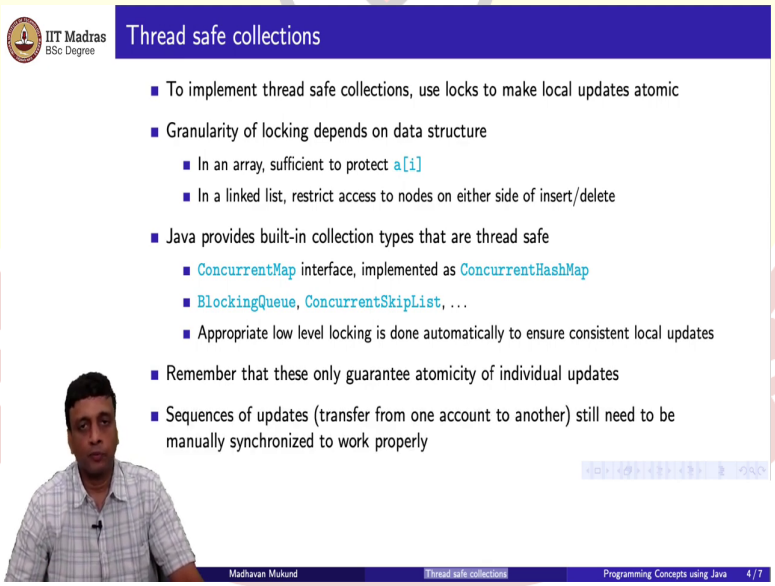
And as usual, this is an interface and there are concrete data structures which implement this thread safe version of the interface. So, for example, concurrent hash map is a particular version of a concurrent map. So, similarly, there is something called a blocking queue. And I

will explain in a minute what a blocking queue is, but it is just a queue. And there are several different ways of implementing blocking queues.

Remember, we said that a queue could be implemented as an array, in which case there is a limit on the length, it could be implemented as a linked list, in which case there is no limit on the length. But then you might have different considerations as to whether this is better or that is better. So, a blocking queue is an abstract, kind of thread safe data structure, which is available and several concrete versions of that, then you have things like concurrent skip lists, and so on.

So, the main thing from our perspective is that these are guaranteed to work. So, an appropriate low-level locking is done automatically to ensure that when we make updates to this. So, for example, in a queue, if I try to insert into a queue, then implicitly I am extending the queue by one position. Now, if these two things overlap, then one of the inserts could get lost. So, what this concurrent thread safe blocking queue will ensure is that if I have two concurrent inserts into a queue, they will both happen in some order.

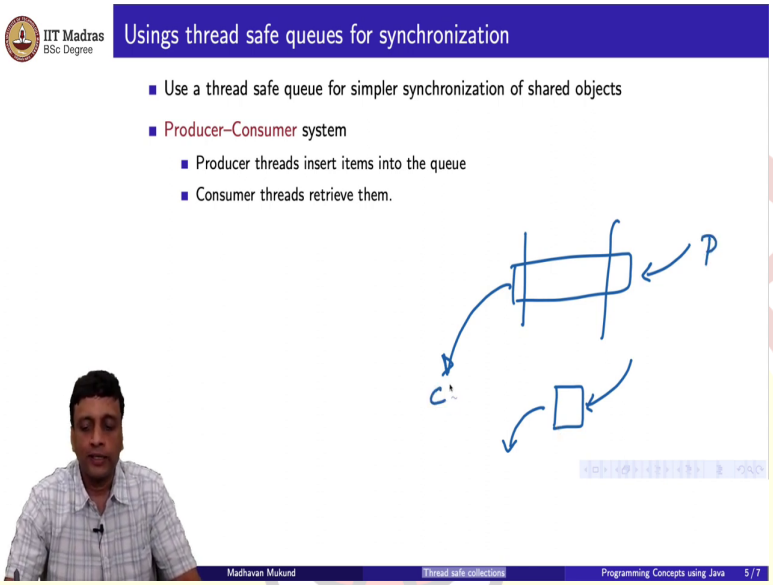
(Refer Slide Time: 13:16)



Again, it is important to remember that this thread, safeness does not resolve the problem that we were looking at earlier, it only makes sure that when we try to update one particular element in that, that works properly. It does not promise us that collection of updates, what we would call a transaction and a database like transferring money from this account to that account requires two updates, one for the source one for the target.

If we want to protect that entire transaction, we still have to put some synchronizing mechanism around the transaction to make sure that nothing else interferes with it. So, we do not get inconsistent updates to the bank account through transfers. So, a sequence of updates must still be synchronized to work properly.

(Refer Slide Time: 13:58)



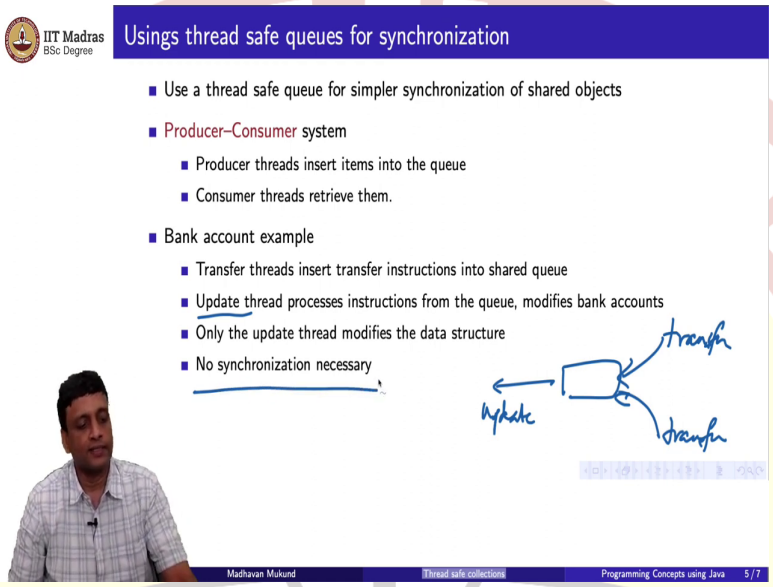
So, it turns out that through the use of these thread safe data structures, we can actually sometimes simplify our overall problem of building with shared data. In particular, a thread safe queue is a very useful device. So, if you have a thread safe queue, remember that two inserts that can people if you add two elements to a queue in parallel from two different threads, you are guaranteed that both elements will get added in some order.

So, with this, what we can do is set up what is called a producer consumer system. So, we have a shared queue, which is thread safe, and we have a producer which is putting things into it. And we have a consumer which is taking things out of it. But because it is thread safe, we are assured that there is never any problem with these two.

So, inserts and deletes anything which involves, so this is see, in general and insert and delete are far apart in a queue because the insert happens at the rear of the queue and the delete happens in the front of the queue. But if I have an empty queue for instance, then if I try to insert something and put an element here, and then I try to remove it, it only works if the insert happens before the remove happens.

So, therefore, there should not be a situation where I do a concurrent insert and delete, and I see something inconsistent about whether the element was put in and removed, or it was never found. So, this is what is guaranteed to us by thread safe queue that this queue behaves well with respect to these positional updates. So, now, this producer consumer system, as I said, a producer puts things in and a consumer takes things out, so how do we use this?

(Refer Slide Time: 15:33)



Well, let us look at our bank account example. What is the problem there? I mean, there is the audit problem. But the main problem is with the updates, if nothing is updated, the audit will never have a problem, the audit is only reading the values. The problem is the transfer is sending money from one account to another account. So, it is distributing the values in the array differently.

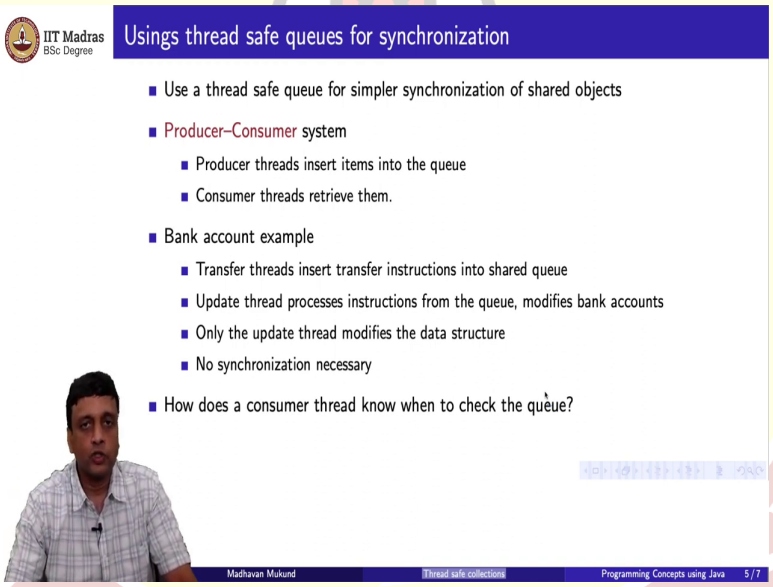
And the problem we have been facing is that different occurrences of transfer and parallel, these updates happen concurrently. And they may be inconsistent updates. So, now, the solution with this thread safe queue is to not allow the transfer to directly upgrade. Instead, you post this item, so you post the transaction for somebody else to process.

So, what happens is that this transfer thread actually says does not say a transfer has happened, it puts into a queue request, saying please transfer money from account I, this much money from account i to account j, another queue will want to transfer from j to k and j is common to both these things, one as a source one that is target. So, that will also say please transfer amount x from j to k.

So, each of these transfer threads will just put this request into the queue. And now, there is an update thread which extracts. So, we have possibly more than one transfer thread, which is putting requests into the queue. But none of this transfer thread is actually accessing the array. So, this queue is then processed by an update thread. So, this update thread, in a natural sense is sequential because it is only one thread. So, it puts out the first transfer finishes, it then picks out the second transfer finishes it and so on.

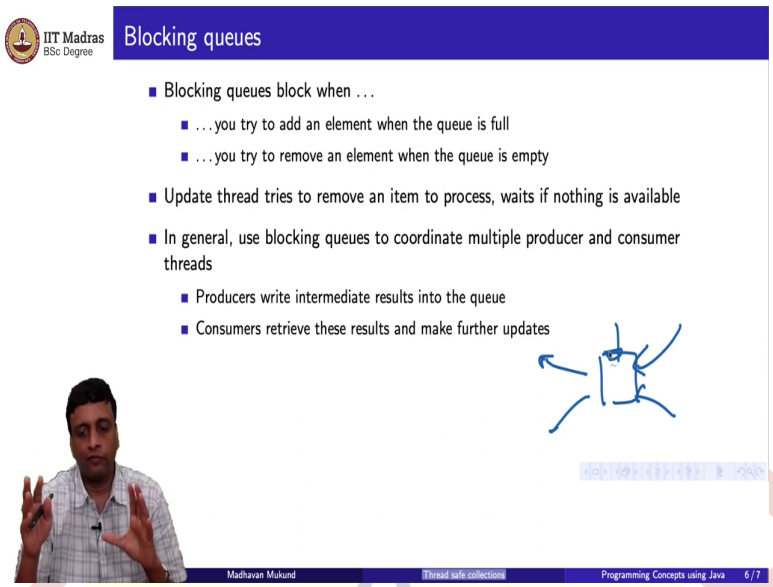
So, this problem of synchronizing access to this bank account array through these transfers is now completely avoided, because the transfers themselves cannot even though they might occur in parallel, they cannot update in parallel, the updates are happening sequentially through the update thread. So, no synchronization is necessary because only the update thread modifies a data structure.

(Refer Slide Time: 17:38)



Now, there is a small question here, which is how does this update thread? The consumer in general know that there is an update way process. How does it actually know that there are things to be done?

(Refer Slide Time: 17:47)



So, it turns out that. that is why we said that Java has what is called a blocking queue. So, a blocking queue allows you to wait for that thing to happen. So, a blocking queue blocks when you try to add an element and the queue is full. So, if you have implemented your queue in an array or something which has a fixed capacity, and there are too many items in the queue already, then you will be allowed to add once something is removed. So, till then the add will block that is you will wait for that thing to occur.

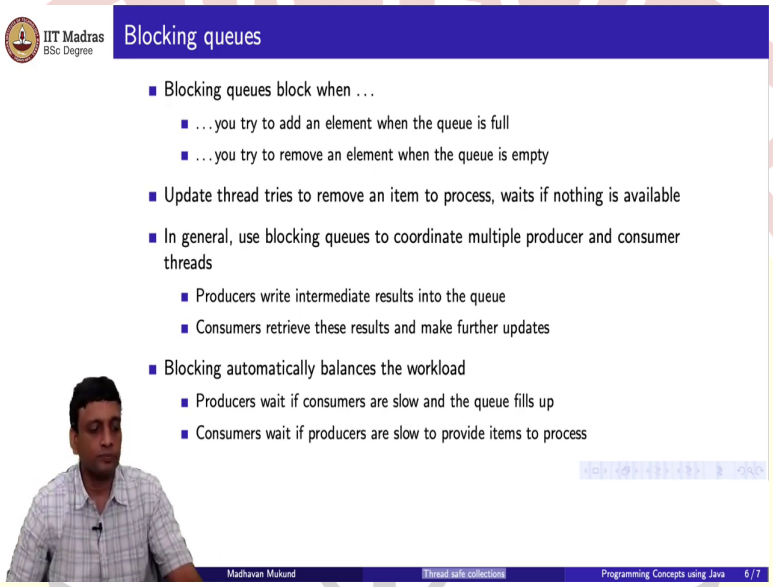
Similarly, if you are trying to remove from a queue, and the queue is empty, you will block until something comes. So, that is what the consumer thread basically does. The update thread tries to remove an item to process. If there is an item process, it will get it, finish it and come back and try it. So, it is a loop where it is constantly trying to remove items to process.

But whenever the like list of items or the queue of items to process is empty, it will block and automatically nothing will happen. So, there is no issue at all, with the fact that the update thread has to know because it always is trying to process and the blocking part of it automatically guarantees.

So, in general, we could have multiple things putting into the queue and multiple things taking out. And we could have a capacity as to how much of pending requests are there. So, we have a fixed size thing. So, the producers are producing some intermediate values, like in this case, they were producing requests saying please transfer something and the consumers were doing some update.

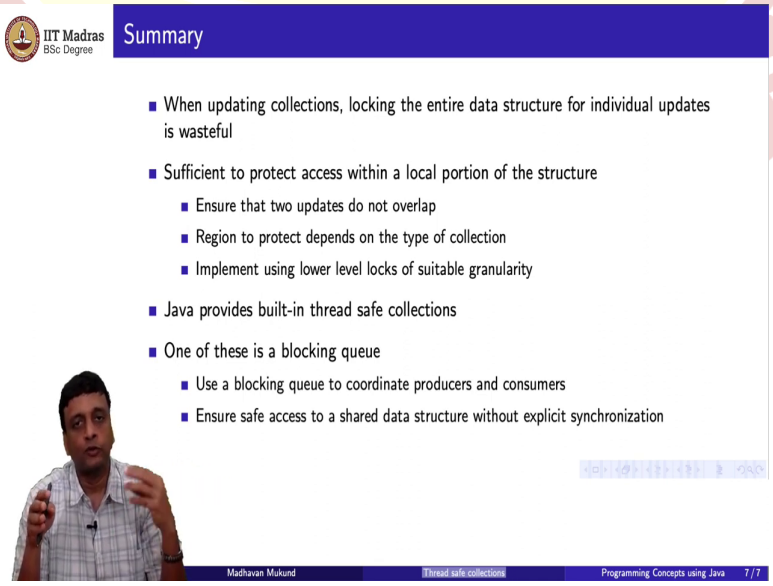
Now, if there are more than one consumer, they have to coordinate between themselves, but between the producers and the consumers, the blocking queue actually coordinate. So, if the consumers are slow, then the producers will find that the queue is full, and they will stop producing until the queue becomes empty, not empty, but it has space. Similarly, if the producers are slower than the consumers will wait. So, then there is an automatic kind of way of balancing out the speed, there is a kind of regulation of this system.

(Refer Slide Time: 19:39)



So, it will the workload of the whole system is automatically balanced because the producers wait if the consumers are slow, and the consumers wait and the producers are slow.

(Refer Slide Time: 19:49)



So, to summarize, what we have seen is that we can always, of course, safely update a collection by locking up the entire collection and making sure that only one update happens to any part of it at a time, but this can hurt performance. So, if we lock up the entire data structure so that we can make an upgrade at one place, we are not optimally using the concurrency that is available to us to speed up throughput in general.

But of course, there could be some overlaps. So, if two threads are trying to access the same element in an array, we should make sure that that happens in a reasonable way so that no update is lost. So, we need to protect this local access to ensure that these local updates do not overlap. And how much we want to protect depends on the data structure, as we saw in the list, in a linked list, you might want to protect 2 or 3 nodes.

And not just at one node, which you are trying to insert or delete, because the entire structures, the list may change in that local region. So, these are implemented by locking, not the entire data structure, by locking a part of the data structure. And that part of the data structure may actually change. Like when you are doing an insert, typically you start at the beginning, and you are searching for the position to insert.

So, you keep locking a small piece and you keep moving the lock. And when you reach the place that you are going to insert, you have locked the nodes that are relevant to that position, and then you insert it there. Now, the nice thing is that Java has these built-in thread safe collections, which do all this locking at a low level without your having to worry about it. So, they provide you with a data structure, where these kinds of individual updates are guaranteed to be thread safe.

But of course, remember that only individual updates are thread safe, the protocol as a whole to coordinate these transactions still has to be designed by you. And in particular, one very useful thread safe data structure is a blocking queue, because this blocking queue allows us to coordinate producers and consumers and set up a mechanism where access to a shared data structure is regulated through the queue.

So, only the consumers of the operation in the queue actually update the data structure. So, there could be several requests to update the queue coming in parallel. But all these updates will be made sequentially and therefore one does not need to explicitly provide a synchronization for the data structure as a whole.